CODING

Gerne. Hier ist der "App-Generations-Prompt", den du direkt verwenden kannst. Ich habe ihn in einem klar definierten Block isoliert, damit du genau weißt, welcher Teil für die KI gedacht ist und welcher Teil unsere Unterhaltung darstellt.-----Hier ist der App-Generations-Prompt (zum Kopieren & Nutzen):
PROMPT START

Du bist ein erfahrener Full-Stack Web Entwickler. Deine Aufgabe ist es, die Web-App "Repository, Rekonstruktion, Run" zu programmieren. Bitte orientiere dich an den folgenden Spezifikationen:

1. App-Zweck & Ziel
Die Web-App dient dazu, Repository-Daten (GitHub-URL oder lokaler Ordner-Upload) zu verarbeiten, zu strukturieren und in verschiedenen Formaten (HTML, JSON, Python, KI-Text) zu exportieren. Die App läuft rein clientseitig im Browser.

2. UI-Design & Layout
- **Front Banner:** Ganz oben prominent der Text: "Repository, Rekonstruktion, Run".
- **Navigation (Linke Sidebar):** Sticky-Menü mit den Punkten: Home, HTML-Repository, JSON-Repository, Python-Struktur, KI-Text-Format, Info.
- **Theme-Engine:** Unterstützung von 3 Modi (Heller Modus, Dunkler Modus, Barrierefrei-Modus mit erhöhtem Kontrast und fetter Schrift). Nutzung von CSS-Variablen. Das System soll sich automatisch an die Browser-Einstellung anpassen.
- **Responsivität:** Adaptives Layout für verschiedene Devices.

3. Funktionalität
- **Import:** GitHub-URL-Eingabefeld sowie ein Button für lokalen Ordner-Upload (File-API, kein Server-Upload nötig).
- **History of Use (Home):** Auf der Startseite, unter der Import-Eingabefläche: Eine Liste der letzten 10 Operationen (Timestamp, Aktion, Modell, Status) gespeichert via LocalStorage.
- **In-App-Hilfe:** Jeder Navigationspunkt soll eine kurze, textliche Kurzanleitung („Wie nutze ich das?“) enthalten.

4. Export-Logik
- **Formate:** HTML, JSON, Python – jeweils mit ZIP-Download-Option (JSZIP).
- **KI-Text-Modul (Kernfunktion):**
  - **Modell-Auswahl:** Dropdown (Gemini, Claude, Grok, TeraAI).
  - **Optimierung:** Die Ausgabe muss spezifisch für das gewählte Modell sein:
    - Automatisches Chunking (Berücksichtigung der jeweiligen Context-Window-Größe/Token-Limits).
    - Automatisches Einfügen von spezifischen System-Instructions für das gewählte Modell.
    - Verwendung Modell-kompatibler Markdown-Code-Blöcke (Syntax-Highlighting).
  - **Output:** Download als .txt-Datei oder Kopieren in die Zwischenablage.

5. Technischer Stack
- Verwende React oder Vue.js für die UI.
- Verwende `JSZip` für ZIP-Generierung.
- Verwende `Prism.js` oder `Highlight.js` für Syntax-Highlighting.
- Nutze `localStorage` für die Geschichte.
- Die App soll vollständig browser-nativ ohne Backend-Server arbeiten (Datenschutz-Fokus).
